

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

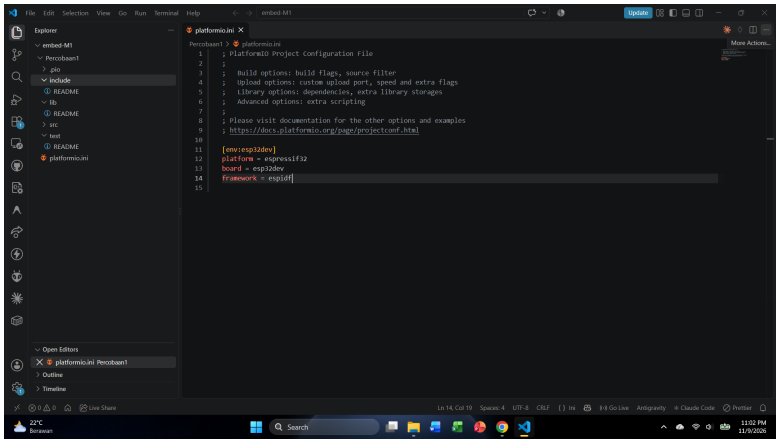
Hafidz Ulum Ramadhani - 5024241014

7 November 2026

1 Langkah-Langkah Percobaan

1.1 Percobaan 1

1. Pastikan proyek Modul 1 sudah dibuat (Bagian D) dan konfigurasi `platformio.ini` sudah sesuai dengan ketentuan.



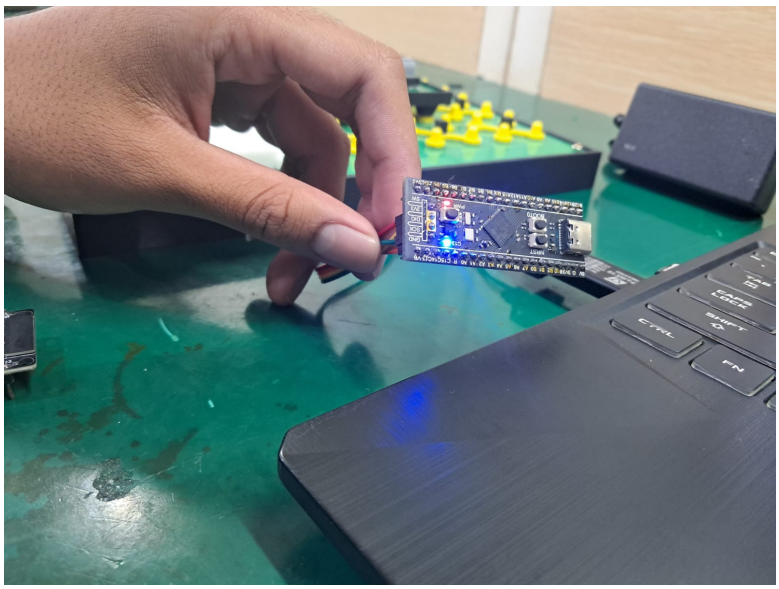
Gambar 1: Konfigurasi proyek Modul 1

2. Hubungkan ST-Link ke board Blackpill menggunakan kabel jumper, kemudian sambungkan ST-Link ke komputer melalui USB. Pastikan setiap pin terhubung sesuai dengan fungsinya, karena posisi pin pada ST-Link dapat berbeda-beda. Berikut merupakan konfigurasi koneksi pin:

Tabel 1: Konfigurasi Koneksi ST-Link dan Blackpill

Pin ST-Link	Pin Blackpill	Fungsi
SWDIO	SWDIO	Jalur data/debug
SWCLK	SWCLK	Jalur clock/debug
GND	GND	Referensi ground bersama
3.3V	3V3	Referensi/tegangan target board

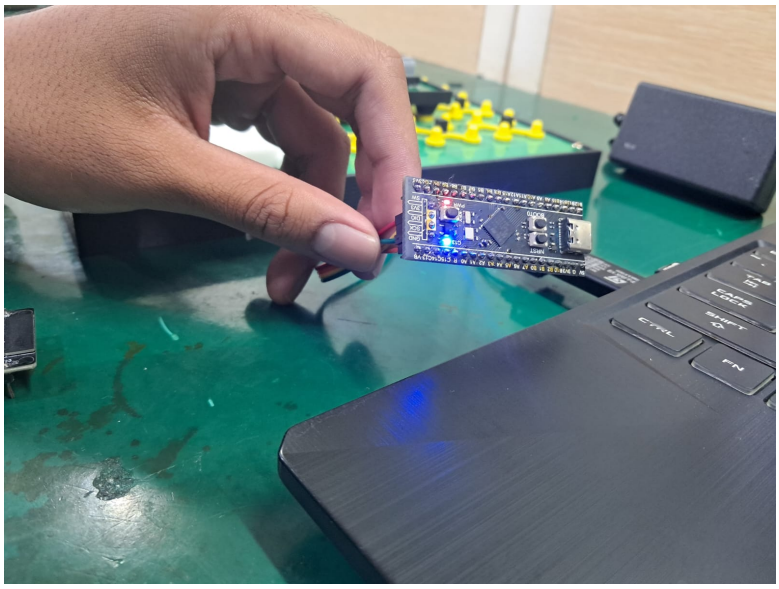
Pastikan ST-Link menggunakan tegangan 3.3V agar sesuai dengan kebutuhan tegangan board Blackpill.



Gambar 2: Koneksi ST-Link dengan board Blackpill

3. Tuliskan kode program *Blink* pada file `src/main.cpp`. Setelah kode selesai ditulis, lakukan proses *Build* untuk memeriksa program, kemudian klik *Upload* untuk mengunggah program ke board Blackpill.

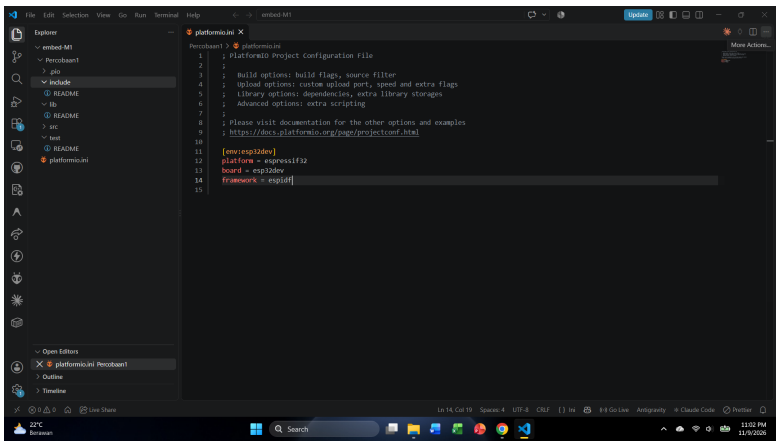
4. Amati LED yang terdapat pada pin PC13 di board Blackpill. Apabila program berhasil dijalankan, LED akan menyala selama setengah detik dan mati selama setengah detik secara berulang.



Gambar 3: Hasil pengujian program Blink pada board Blackpill

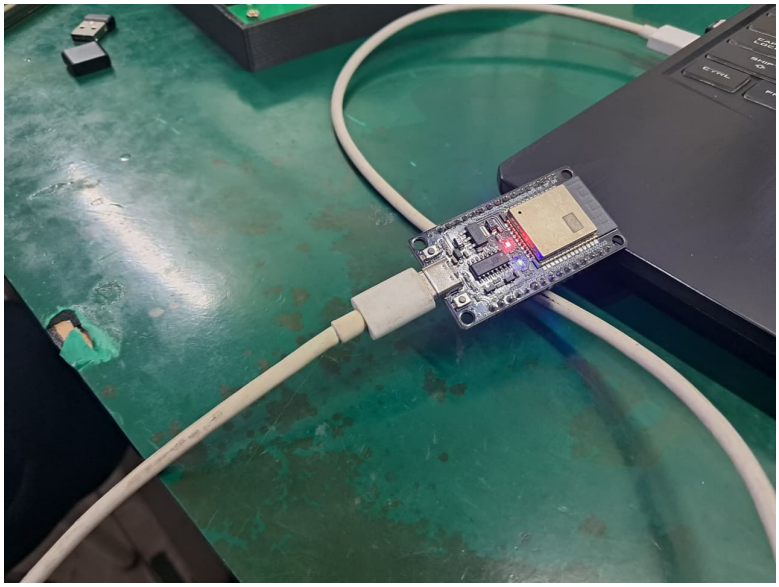
1.2 Percobaan 2

1. Ganti isi file `platformio.ini` dengan konfigurasi yang telah ditentukan, yaitu menggunakan board ESP32 DevKit dengan framework ESP-IDF.



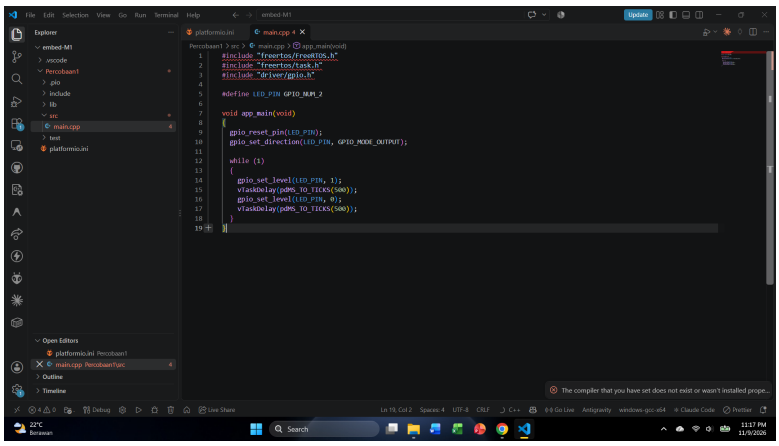
Gambar 4: Konfigurasi platformio.ini menggunakan ESP-IDF

2. Hubungkan board ESP32 ke komputer menggunakan kabel USB. Selanjutnya, pastikan komputer dapat mengenali port komunikasi yang digunakan oleh board ESP32.



Gambar 5: Deteksi port ESP32 pada komputer

3. Hapus file `src/main.cpp`, kemudian buat file baru dengan nama `src/main.c`. Tuliskan kode program *Blink* yang telah disediakan. Setelah itu, lakukan proses *Build* untuk memeriksa program dan *Upload* untuk mengunggah program ke board ESP32.



Gambar 6: Penulisan kode Blink dan proses Upload pada ESP32

4. Lakukan analisis mengenai perbedaan pemrograman menggunakan ESP-IDF dan Arduino.

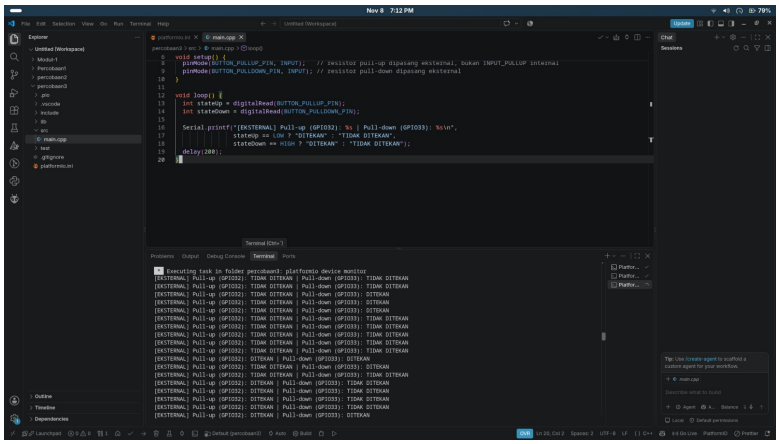
   Analisis:

   Pemrograman menggunakan ESP-IDF dan Arduino memiliki perbedaan dari segi tingkat abstraksi, struktur program, serta pengelolaan perangkat keras. Arduino menyediakan berbagai fungsi dan library yang lebih sederhana sehingga proses pemrograman menjadi lebih mudah, terutama bagi pemula. Sementara itu, ESP-IDF merupakan framework resmi dari Espressif yang memberikan kontrol lebih luas terhadap perangkat keras dan fitur ESP32.

   Pada Arduino, program umumnya menggunakan fungsi `setup()` dan `loop()`, sedangkan ESP-IDF menggunakan fungsi `app_main()` sebagai titik awal eksekusi program. ESP-IDF juga menyediakan pengaturan yang lebih terperinci, seperti konfigurasi GPIO, task, dan manajemen sistem. Dengan demikian, Arduino lebih praktis untuk pengembangan aplikasi sederhana, sedangkan ESP-IDF lebih fleksibel untuk pengembangan sistem yang membutuhkan kontrol dan konfigurasi perangkat secara lebih mendalam.

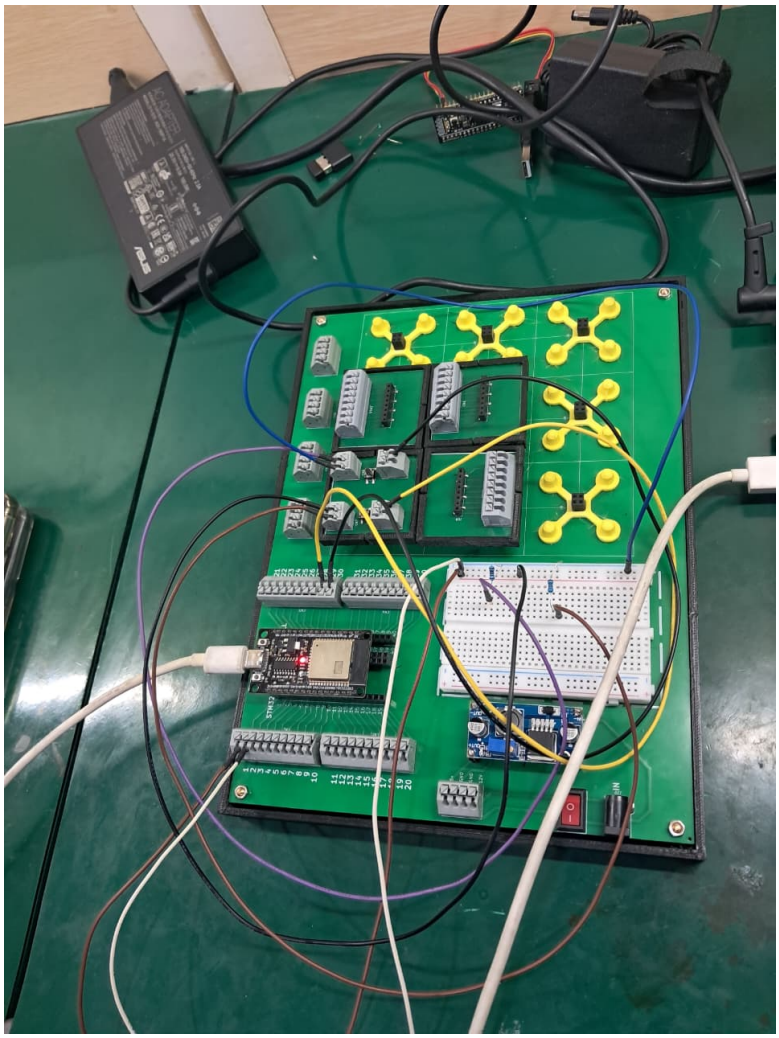
1.3 Percobaan 3

1. Ganti isi file `platformio.ini` kembali ke konfigurasi Arduino seperti sebelumnya. Hapus file `src/main.c`, kemudian buat kembali file `src/main.cpp`.



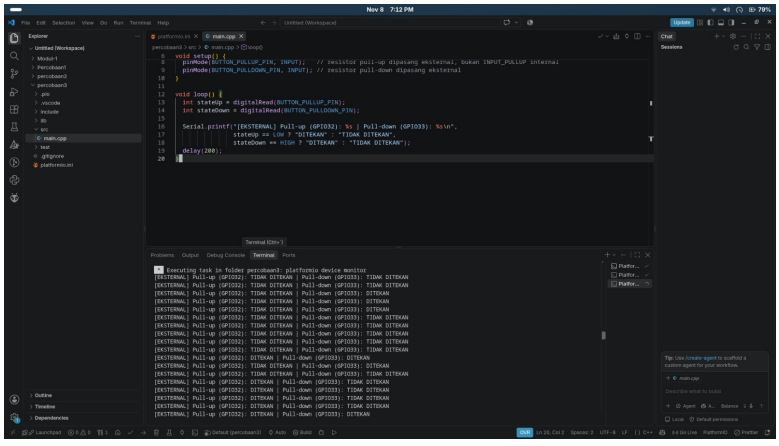
Gambar 7: Konfigurasi PlatformIO menggunakan framework Arduino

2. Pasang dua buah tombol pada GPIO 32 dan GPIO 33, lengkap dengan resistor sesuai dengan Skema Bagian A.



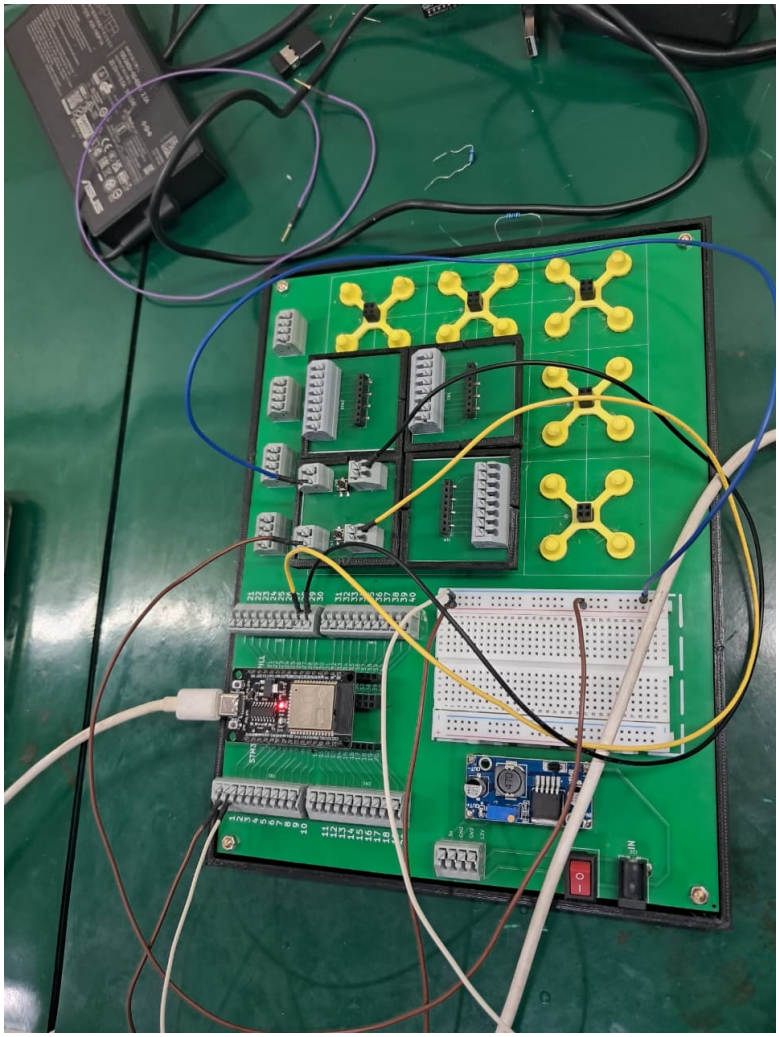
Gambar 8: Rangkaian dua tombol dengan resistor eksternal

3. Tuliskan **Kode Program A (Eksternal)** pada file `src/main.cpp`. Selanjutnya, klik *Build* dan *Upload*. Setelah program berhasil diunggah, tekan kedua tombol dan catat hasil yang ditampilkan pada *Serial Monitor*.



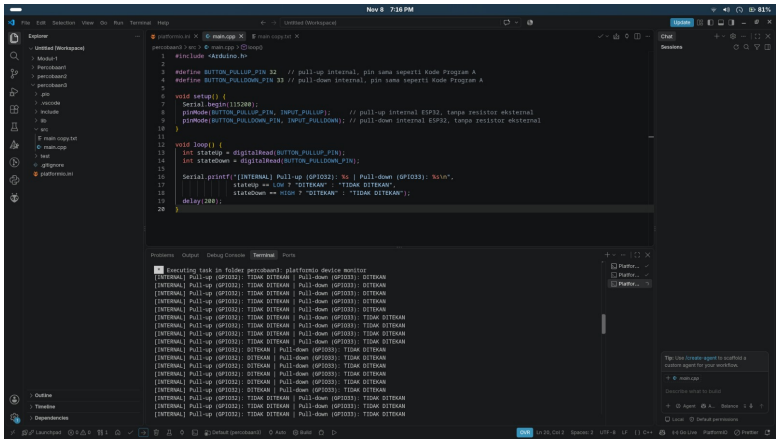
Gambar 9: Pengujian tombol menggunakan resistor eksternal

4. Lepaskan kedua resistor sesuai dengan Skema Bagian B (Gambar 7), sementara kedua tombol tetap terpasang. Perhatikan Gambar 8 dan Gambar 9 untuk memastikan pasangan kaki tombol yang digunakan sudah benar.



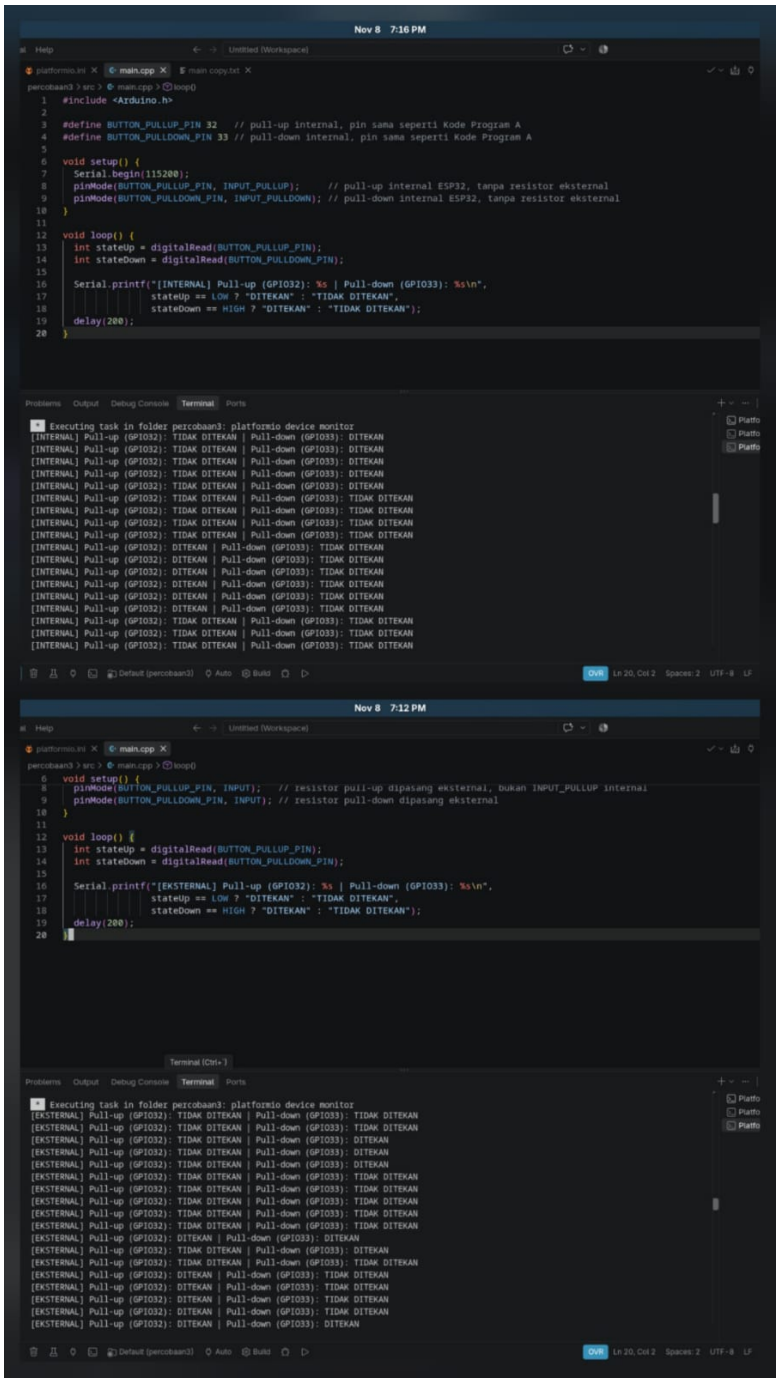
Gambar 10: Rangkaian tombol tanpa resistor eksternal

5. Ganti isi file `src/main.cpp` dengan **Kode Program B (Internal)** yang telah disediakan. Kemudian, klik *Build* dan *Upload* untuk menjalankan program pada board ESP32.



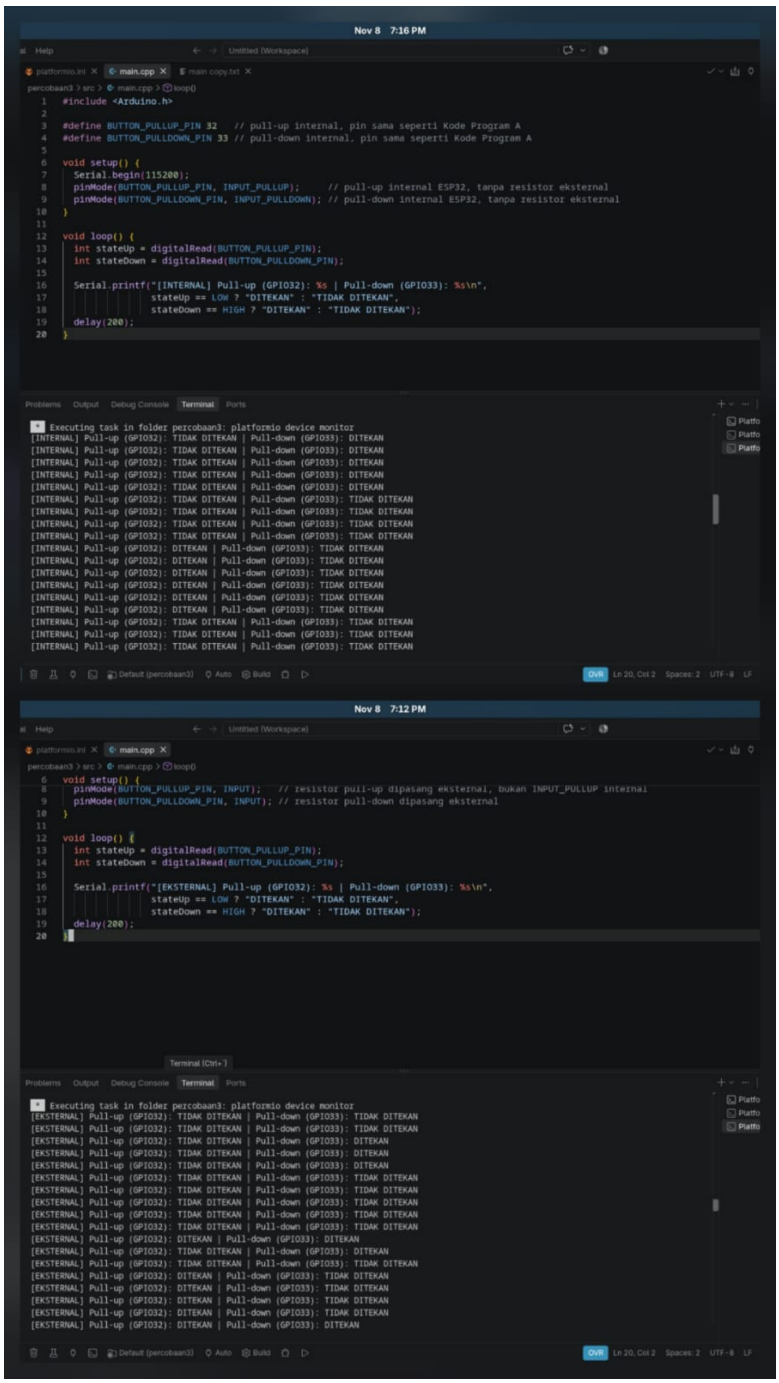
Gambar 11: Implementasi program menggunakan internal pull-up dan pull-down

6. Setelah program berhasil diunggah, tekan kedua tombol seperti pada pengujian sebelumnya. Amati dan catat hasil yang ditampilkan pada *Serial Monitor*.



Gambar 12: Hasil pengujian tombol menggunakan resistor internal

7. Bandingkan hasil pengujian Program A dan Program B. Hasil yang diperoleh seharusnya sama, yaitu pada konfigurasi *pull-up*, nilai GPIO adalah HIGH ketika tombol tidak ditekan dan LOW ketika tombol ditekan. Sebaliknya, pada konfigurasi *pull-down*, nilai GPIO adalah LOW ketika tombol tidak ditekan dan HIGH ketika tombol ditekan.

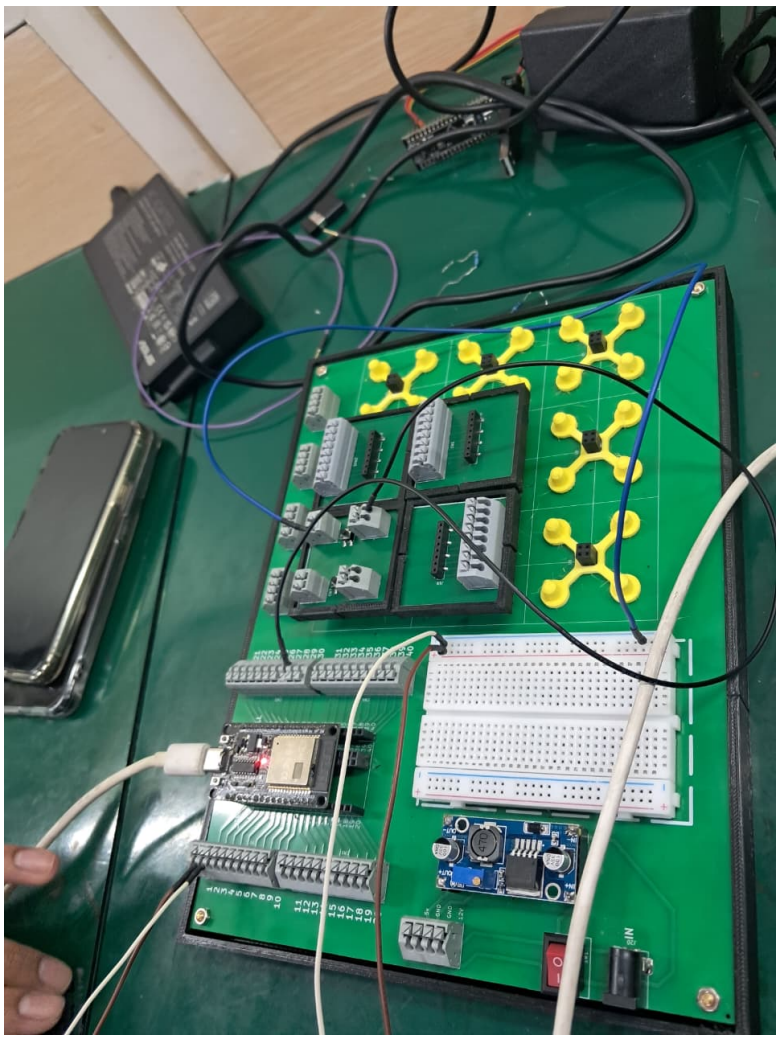


Gambar 13: Perbandingan hasil pengujian Program A dan Program B

8. Apabila hasil pengujian menunjukkan kondisi yang terbalik atau berbeda, periksa kembali rangkaian dan pastikan kedua resistor eksternal sudah benar-benar dilepas.

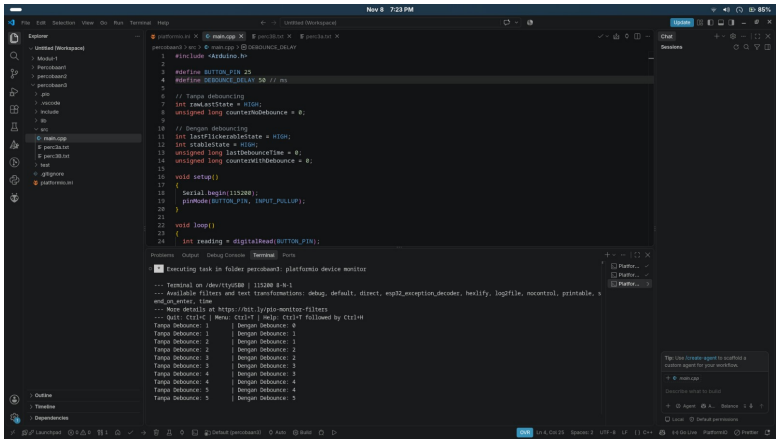
1.4 Percobaan 4

1. Pasang tombol pada rangkaian sesuai dengan skema yang telah diberikan.



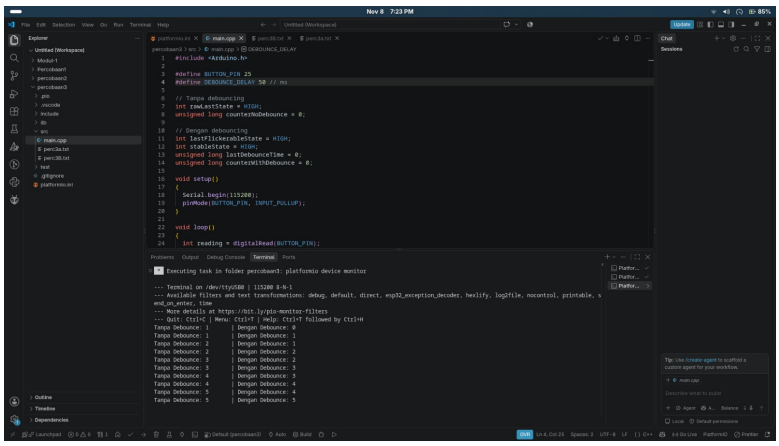
Gambar 14: Rangkaian tombol sesuai skema

2. Tuliskan kode program yang telah disediakan. Program ini memiliki dua penghitung, yaitu `counterNoDebounce` (tanpa penyaring getaran tombol) dan `counterWithDebounce` (dengan penyaring getaran tombol).



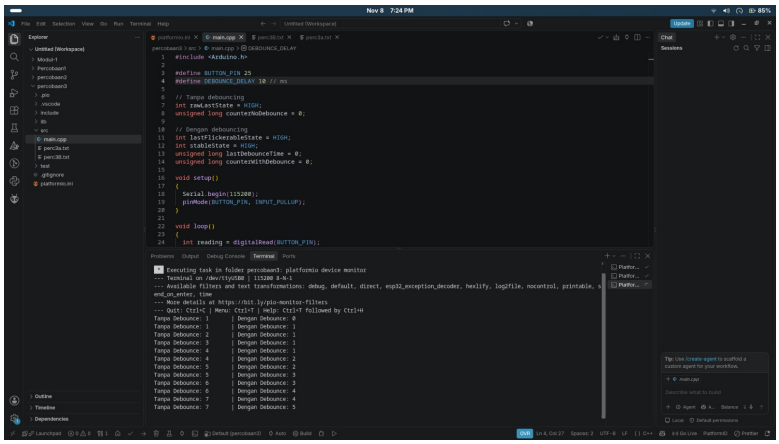
Gambar 15: Implementasi program debounce pada ESP32

3. Tekan tombol sebanyak satu kali, kemudian bandingkan nilai kedua penghitung yang ditampilkan pada *Serial Monitor*.



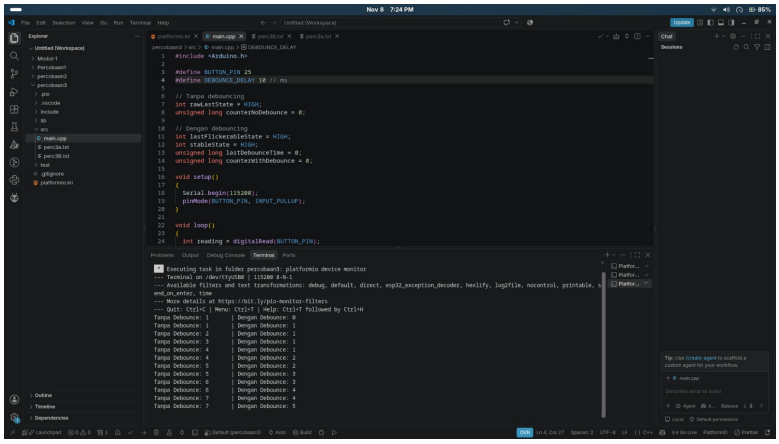
Gambar 16: Hasil pengujian kedua penghitung pada Serial Monitor

4. Ulangi pengujian dengan variasi cara menekan tombol, baik secara cepat maupun lambat. Penghitung `counterNoDebounce` dapat bertambah lebih dari satu untuk satu kali tekanan, sedangkan `counterWithDebounce` seharusnya bertambah tepat satu. Apabila kedua penghitung menunjukkan hasil yang sama, lakukan pengujian kembali karena getaran tombol tidak selalu muncul.



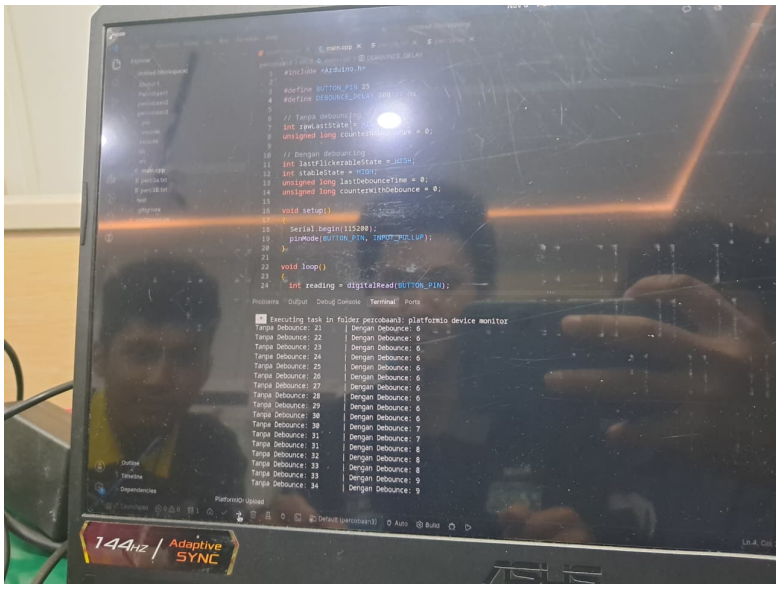
Gambar 17: Pengujian debounce dengan variasi tekanan tombol

5. Ubah nilai `DEBOUNCE_DELAY` menjadi `10` (10 ms). Setelah itu, klik *Build* dan *Upload*, kemudian lakukan pengujian kembali dengan menekan tombol.



Gambar 18: Pengujian dengan nilai debounce delay 10 ms

6. Ubah nilai `DEBOUNCE_DELAY` menjadi `200` (200 ms). Selanjutnya, klik *Build* dan *Upload*, kemudian tekan tombol secara cepat dan berulang kali.

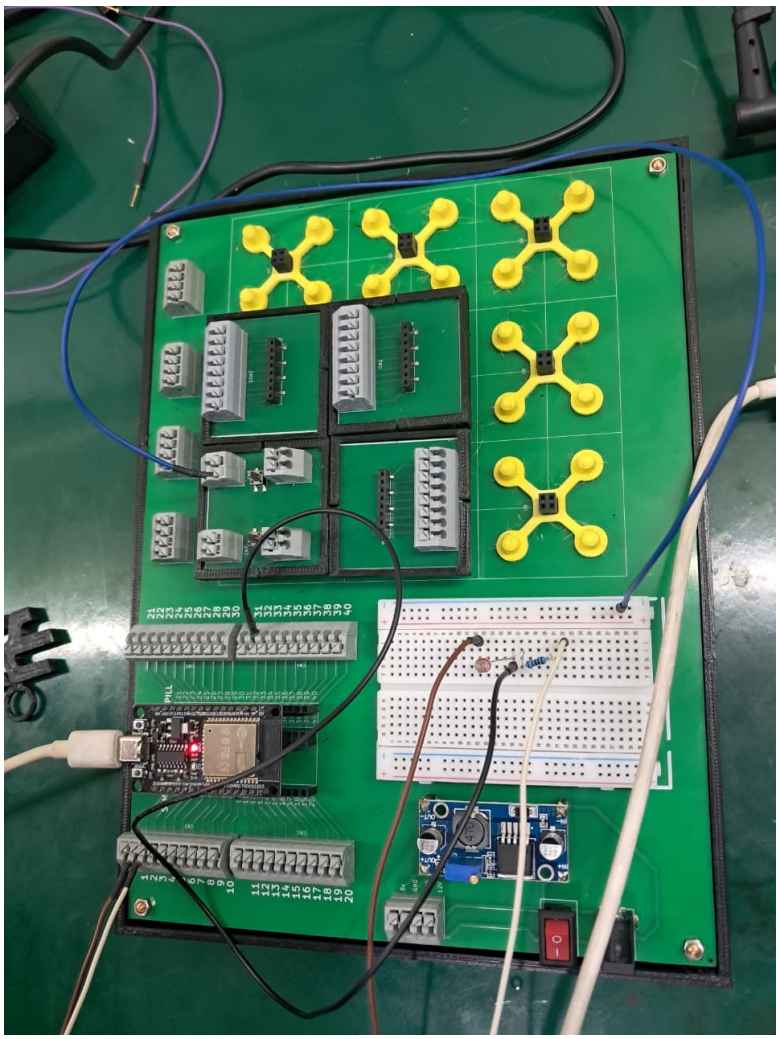


Gambar 19: Pengujian dengan nilai debounce delay 200 ms

7. Bandingkan hasil pengujian menggunakan tiga nilai `DEBOUNCE_DELAY`, yaitu 10 ms, 50 ms, dan 200 ms. Analisis kelebihan dan kekurangan dari setiap nilai berdasarkan kemampuan menyaring getaran tombol dan kecepatan respons tombol.

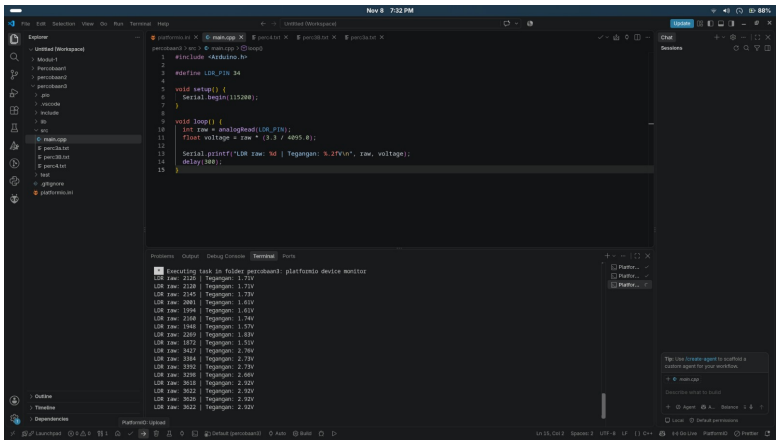
1.5 Percobaan 5

1. Pasang komponen LDR bersama resistor sesuai dengan skema rangkaian yang telah diberikan. Hubungkan titik tengah antara LDR dan resistor ke pin GPIO 34 pada board ESP32.



Gambar 20: Rangkaian LDR dan resistor pada ESP32

2. Tuliskan program yang telah disediakan untuk membaca nilai ADC mentah dari sensor LDR dan mengonversinya menjadi nilai tegangan.

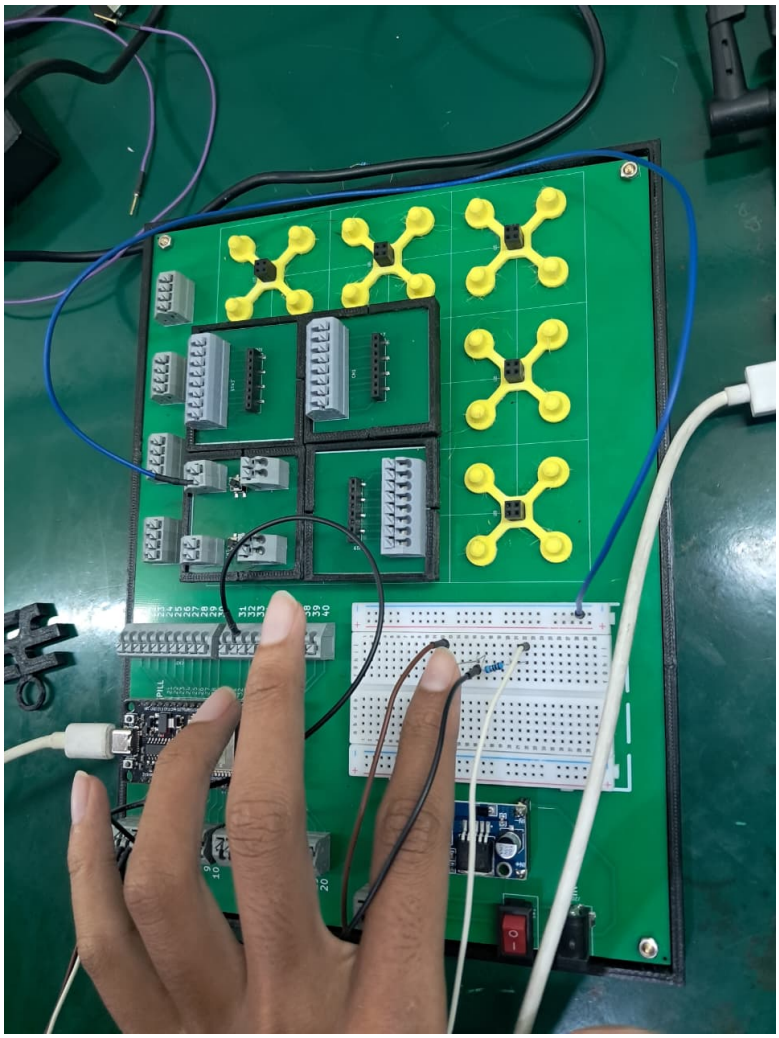


Gambar 21: Implementasi program pembacaan sensor LDR

3. Klik *Build* untuk memeriksa program dan *Upload* untuk mengunggah program ke board ESP32. Setelah proses selesai, buka *Serial Monitor* untuk melihat hasil pembacaan sensor.

4. Tutup permukaan LDR menggunakan tangan, kemudian arahkan lampu ke sensor. Amati per-

ubahan nilai ADC mentah yang ditampilkan pada *Serial Monitor*, dengan rentang pembacaan antara 0 hingga 4095.

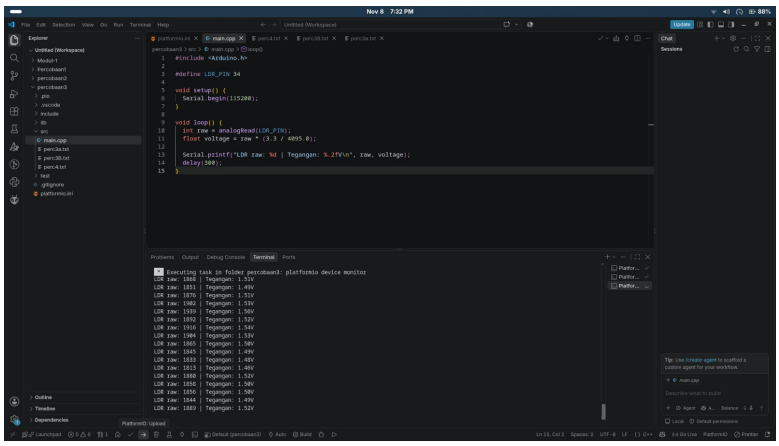


Gambar 22: Pengujian sensor LDR pada kondisi gelap dan terang

5. Lakukan analisis mengenai alasan sinyal analog dapat menunjukkan berbagai kondisi, tidak hanya HIGH atau LOW.

   Analisis:

   Sinyal analog dapat menunjukkan banyak kondisi karena memiliki nilai yang bervariasi dalam suatu rentang tegangan, tidak hanya terbatas pada dua kondisi seperti sinyal digital. Pada ESP32, sinyal analog yang diterima melalui ADC dikonversi menjadi nilai digital dengan rentang pembacaan 0 hingga 4095. Perubahan intensitas cahaya yang diterima LDR akan memengaruhi nilai resistansinya sehingga tegangan pada titik pembacaan juga mengalami perubahan. Oleh karena itu, ketika LDR berada dalam kondisi gelap maupun terang, nilai ADC yang dihasilkan dapat berbeda-beda sesuai dengan intensitas cahaya yang diterima. Hal ini memungkinkan sensor analog mendeteksi perubahan kondisi secara lebih detail dibandingkan sinyal digital yang hanya memiliki dua kondisi, yaitu HIGH dan LOW.



Gambar 23: Hasil pengamatan perubahan nilai ADC sensor LDR

2 Analisis Hasil Percobaan

2.1 Percobaan 1

Analisis Percobaan 1: Proses flashing program Blink ke board STM32 Blackpill (STM32F401CCU6) menggunakan framework Arduino (STM32duino) melalui programmer ST-Link dengan protokol SWD (Serial Wire Debug) berjalan dengan sukses tanpa mengalami kendala. Proses komunikasi antara komputer dan mikrokontroler dilakukan melalui jalur SWDIO, SWCLK, GND, dan 3.3V yang berfungsi sebagai jalur komunikasi serta referensi tegangan. Keberhasilan proses flashing menunjukkan bahwa konfigurasi board, koneksi programmer, serta lingkungan pengembangan telah berjalan dengan baik.

Berdasarkan hasil pengujian, penggunaan fungsi `digitalWrite(PC13, LOW)` dan `delay(500)` berhasil mengendalikan LED onboard pada pin PC13 sehingga dapat menyala dan mati secara bergantian dengan interval waktu 500 ms. LED pada pin tersebut memiliki karakteristik aktif LOW, sehingga kondisi logika LOW menyebabkan LED menyala, sedangkan kondisi HIGH menyebabkan LED mati. Hal ini menunjukkan bahwa pengendalian keluaran digital pada mikrokontroler dapat dilakukan melalui pengaturan kondisi logika pin.

Selain itu, penggunaan fungsi `delay()` menunjukkan bahwa program menjalankan instruksi secara berurutan dengan memberikan jeda waktu tertentu sebelum melanjutkan instruksi berikutnya. Dengan demikian, percobaan ini membuktikan bahwa framework Arduino (STM32duino) mampu menyederhanakan proses pemrograman perangkat keras STM32 melalui fungsi-fungsi yang lebih mudah digunakan. Secara keseluruhan, hasil percobaan menunjukkan bahwa proses pemrograman, komunikasi melalui ST-Link, dan pengendalian LED onboard telah berhasil dilakukan pada mikrokontroler STM32F401CCU6 berbasis arsitektur ARM Cortex-M4.

2.2 Percobaan 2

Analisis Percobaan 2: Proses pengunggahan dan eksekusi program Blink pada ESP32 DevKit menggunakan framework native ESP-IDF (Espressif IoT Development Framework) berhasil dilakukan tanpa mengalami kendala. Berbeda dengan framework Arduino yang menyediakan abstraksi fungsi pemrograman tingkat tinggi, ESP-IDF memberikan akses yang lebih langsung terhadap peripheral perangkat keras ESP32 melalui berbagai library dan API yang telah disediakan. Keberhasilan proses kompilasi dan pengunggahan menunjukkan bahwa konfigurasi ESP-IDF serta komunikasi antara

komputer dan board ESP32 telah berjalan dengan baik.

Program dijalankan melalui fungsi `app_main(void)` sebagai titik masuk utama aplikasi pada lingkungan ESP-IDF. Pengaturan kondisi keluaran digital dilakukan menggunakan fungsi `gpio_set_level(GPIO_NUM_1 1)` untuk memberikan logika HIGH pada GPIO 2, sedangkan logika LOW digunakan untuk mengubah kondisi keluaran menjadi rendah. Proses pergantian kondisi tersebut menghasilkan aktivitas LED yang menyala dan mati secara bergantian. Pengaturan jeda waktu dilakukan menggunakan fungsi `vTaskDelay(pdMS_TO_TICKS(500))` yang memberikan penundaan selama kurang lebih 500 ms berdasarkan sistem tick FreeRTOS.

Penggunaan fungsi `vTaskDelay()` menunjukkan bahwa ESP32 memanfaatkan mekanisme penjadwalan tugas dari FreeRTOS. Berbeda dengan fungsi `delay()` pada Arduino yang memberikan penundaan dalam alur program, `vTaskDelay()` memungkinkan task yang sedang berjalan untuk berada dalam kondisi tertunda sehingga task lain dapat memperoleh kesempatan untuk dieksekusi oleh scheduler. Hal ini menjadi salah satu keunggulan penggunaan ESP-IDF dalam pengembangan sistem embedded yang membutuhkan pengelolaan beberapa proses secara bersamaan.

Dengan demikian, percobaan ini menunjukkan bahwa pengendalian GPIO pada ESP32 dapat dilakukan melalui framework native ESP-IDF dengan memanfaatkan API yang lebih spesifik terhadap perangkat keras. Hasil pengujian juga memberikan pemahaman mengenai perbedaan pendekatan pemrograman antara framework Arduino dan ESP-IDF, khususnya dalam hal pengaturan GPIO, manajemen waktu, dan pemanfaatan sistem operasi real-time FreeRTOS.

2.3 Percobaan 3

Analisis Percobaan 3: Penggunaan konfigurasi resistor pull-up dan pull-down pada ESP32 berhasil menunjukkan perbedaan kondisi logika pin digital berdasarkan keadaan tombol yang terhubung pada rangkaian. Konfigurasi ini dapat dilakukan dengan menggunakan resistor eksternal maupun memanfaatkan resistor internal yang telah tersedia pada mikrokontroler melalui pengaturan `pinMode(pin, INPUT_PULLUP)` dan `pinMode(pin, INPUT_PULLDOWN)`. Kedua konfigurasi tersebut berfungsi untuk memberikan kondisi logika awal yang pasti pada pin input ketika tombol tidak sedang ditekan.

Pada konfigurasi pull-up, pin input akan terhubung secara internal ke sumber tegangan 3.3V sehingga menghasilkan kondisi HIGH ketika tombol tidak ditekan. Sebaliknya, ketika tombol ditekan dan menghubungkan pin ke GND, kondisi logika akan berubah menjadi LOW. Sementara itu, pada konfigurasi pull-down, pin input akan berada pada kondisi LOW ketika tombol tidak ditekan karena terhubung ke GND melalui resistor. Ketika tombol ditekan dan menghubungkan pin ke sumber tegangan 3.3V, kondisi logika akan berubah menjadi HIGH.

Berdasarkan hasil pengujian, kedua konfigurasi tersebut mampu menghasilkan pembacaan kondisi digital yang lebih stabil dan konsisten. Keberadaan resistor pull-up maupun pull-down sangat penting untuk mencegah kondisi floating, yaitu keadaan ketika pin input tidak memiliki referensi tegangan yang jelas. Kondisi floating dapat menyebabkan pin membaca perubahan logika secara tidak menentu akibat pengaruh noise atau gangguan listrik dari lingkungan sekitar. Akibatnya, mikrokontroler dapat mendeteksi perubahan kondisi input meskipun tombol sebenarnya tidak sedang ditekan.

Dengan demikian, percobaan ini membuktikan bahwa konfigurasi pull-up dan pull-down merupakan bagian penting dalam perancangan rangkaian input digital. Pemilihan konfigurasi yang tepat dapat membantu menjaga kestabilan pembacaan sinyal, mengurangi kemungkinan kesalahan pembacaan, serta memastikan mikrokontroler dapat membedakan kondisi tombol ditekan dan tidak ditekan secara konsisten.

2.4 Percobaan 4

Analisis Percobaan 4: Pengujian teknik debouncing pada rangkaian pushbutton menunjukkan bahwa perubahan kondisi mekanis pada kontak tombol dapat menghasilkan beberapa perubahan sinyal digital dalam waktu yang sangat singkat. Fenomena tersebut dikenal sebagai bouncing, yaitu kondisi ketika kontak logam di dalam tombol mengalami getaran atau pantulan sesaat ketika tombol ditekan maupun dilepaskan. Akibatnya, sinyal yang seharusnya hanya mengalami satu kali perubahan dari HIGH ke LOW atau sebaliknya dapat menghasilkan beberapa transisi logika sebelum akhirnya mencapai kondisi stabil.

Berdasarkan hasil pengujian, pembacaan tombol tanpa penerapan debouncing dapat menyebabkan terjadinya kesalahan deteksi penekanan. Mikrokontroler berpotensi membaca satu kali penekanan fisik sebagai beberapa kali perubahan kondisi input. Hal tersebut dapat menyebabkan nilai counter meningkat lebih dari satu kali meskipun tombol hanya ditekan sekali. Pada pengujian yang dilakukan, kondisi tersebut ditunjukkan dengan nilai counter yang dapat meningkat hingga $n > 7$ akibat beberapa perubahan sinyal yang terbaca selama proses bouncing.

Untuk mengatasi permasalahan tersebut, digunakan teknik debouncing berbasis waktu dengan memanfaatkan fungsi `millis()`. Program mencatat waktu ketika terjadi perubahan pembacaan tombol, kemudian menunggu hingga kondisi input tidak mengalami perubahan selama interval tertentu. Pada percobaan ini digunakan nilai DEBOUNCE_DELAY $= 50$ ms sebagai batas waktu untuk memastikan bahwa kondisi tombol telah stabil sebelum perubahan tersebut dianggap sebagai penekanan yang valid.

Hasil pengujian menunjukkan bahwa penerapan jeda stabilitas selama 50 ms mampu mengurangi perubahan sinyal palsu akibat bouncing. Dengan metode tersebut, program hanya memproses perubahan kondisi tombol setelah sinyal berada dalam keadaan stabil, sehingga pembacaan menjadi lebih akurat dibandingkan dengan pembacaan langsung tanpa penyaringan. Penggunaan `millis()` juga memungkinkan proses debouncing dilakukan tanpa menghentikan keseluruhan eksekusi program melalui fungsi penundaan seperti `delay()`.

Dengan demikian, percobaan ini membuktikan bahwa debouncing diperlukan dalam sistem yang menggunakan pushbutton sebagai perangkat input digital. Penerapan teknik tersebut dapat meningkatkan keandalan pembacaan tombol, mencegah terjadinya deteksi penekanan berulang yang tidak diinginkan, serta memastikan bahwa perubahan kondisi input yang diproses oleh mikrokontroler benar-benar merepresentasikan tindakan pengguna.

2.5 Percobaan 5

Analisis Percobaan 5: Pengujian sensor LDR (Light Dependent Resistor) menggunakan ESP32 menunjukkan bahwa perubahan intensitas cahaya dapat dideteksi melalui perubahan nilai resistansi sensor yang kemudian dikonversi menjadi perubahan tegangan menggunakan rangkaian pembagi tegangan. Titik keluaran dari rangkaian tersebut dihubungkan ke GPIO 34 ESP32 yang mendukung fungsi ADC (Analog-to-Digital Converter). Dengan konfigurasi tersebut, mikrokontroler dapat membaca perubahan tegangan yang dihasilkan oleh sensor LDR dan mengubahnya menjadi nilai digital yang dapat diproses oleh program.

Berdasarkan hasil pengujian, ADC 12-bit pada ESP32 mampu menghasilkan nilai pembacaan dalam rentang 0 hingga 4095. Nilai tersebut merepresentasikan besarnya tegangan analog yang diterima oleh pin ADC. Ketika intensitas cahaya yang diterima LDR berubah, resistansi sensor juga

mengalami perubahan sehingga tegangan keluaran pada rangkaian pembagi tegangan ikut berubah. Perubahan tegangan tersebut kemudian menghasilkan nilai ADC yang berbeda. Dengan demikian, semakin besar perubahan kondisi pencahayaan, semakin besar pula kemungkinan terjadinya perubahan nilai pembacaan ADC.

Untuk mengetahui nilai tegangan berdasarkan hasil pembacaan ADC, digunakan persamaan konversi tegangan sebagai berikut:

$$V = \text{raw} \times \left(\frac{3.3}{4095.0} \right)$$

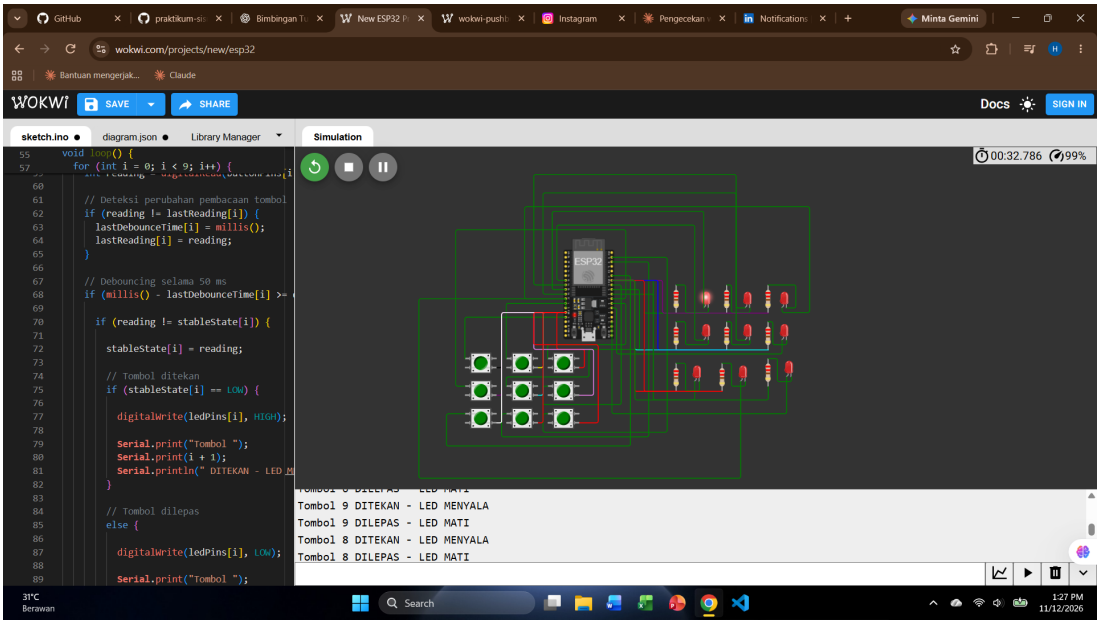
Pada persamaan tersebut, variabel raw merupakan nilai hasil pembacaan ADC, sedangkan 3.3V merupakan tegangan referensi yang digunakan sebagai pendekatan dalam perhitungan. Hasil konversi tersebut memberikan gambaran mengenai besarnya tegangan yang diterima oleh pin GPIO 34 berdasarkan nilai ADC yang diperoleh.

Berbeda dengan digital input yang hanya memiliki dua kemungkinan kondisi logika, yaitu HIGH dan LOW, pembacaan analog melalui ADC memungkinkan mikrokontroler memperoleh lebih banyak variasi nilai. Hal ini disebabkan ADC melakukan proses kuantisasi dengan membagi rentang tegangan masukan menjadi sejumlah tingkatan nilai digital. Pada resolusi 12-bit terdapat $2^{12} = 4096$ kemungkinan nilai, yaitu mulai dari 0 hingga 4095. Oleh karena itu, perubahan intensitas cahaya pada LDR dapat direpresentasikan dalam bentuk nilai digital yang lebih beragam dibandingkan dengan pembacaan digital biasa.

Secara keseluruhan, percobaan ini menunjukkan bahwa penggunaan ADC memungkinkan ESP32 membaca besaran fisik yang memiliki perubahan kontinu, seperti intensitas cahaya, melalui perubahan tegangan listrik. Hasil pengujian juga memperlihatkan bahwa sensor LDR dapat digunakan sebagai perangkat input analog untuk mendukung berbagai aplikasi sistem embedded, misalnya sistem lampu otomatis, pemantauan intensitas cahaya, dan pengaturan pencahayaan berdasarkan kondisi lingkungan.

3 Hasil Tugas Modul

Praktikum: Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3



Gambar 24: Hasil Simulasi Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

Source Code

Listing 1: Program Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3

```
#include <Arduino.h>

// Pin Pushbutton
const int buttonPins[9] = {
  13, 14, 16,
  17, 18, 19,
  21, 22, 23
};

// Pin LED
const int ledPins[9] = {
  25, 26, 27,
  32, 33, 4,
  5, 12, 15
};

// Variabel Debouncing
int lastReading[9];
int stableState[9];

unsigned long lastDebounceTime[9] = {0};

const unsigned long debounceDelay = 50;

// SETUP
```

```
void setup() {
  Serial.begin(115200);

  for (int i = 0; i < 9; i++) {
    pinMode(buttonPins[i], INPUT_PULLUP);
    pinMode(ledPins[i], OUTPUT);

    digitalWrite(ledPins[i], LOW);

    lastReading[i] = HIGH;
    stableState[i] = HIGH;
  }

  Serial.println("Matrix Pushbutton 3x3 Ready!");
}

// LOOP
void loop() {
  for (int i = 0; i < 9; i++) {
    int reading = digitalRead(buttonPins[i]);

    // Deteksi perubahan pembacaan tombol
    if (reading != lastReading[i]) {
      lastDebounceTime[i] = millis();
      lastReading[i] = reading;
    }

    // Debouncing selama 50 ms
    if (millis() - lastDebounceTime[i] >= debounceDelay) {
      if (reading != stableState[i]) {
        stableState[i] = reading;

        // Tombol ditekan
        if (stableState[i] == LOW) {
          digitalWrite(ledPins[i], HIGH);

          Serial.print("Tombol ");
          Serial.print(i + 1);
          Serial.println(" DITEKAN - LED MENYALA");
        }

        // Tombol dilepas
        else {
          digitalWrite(ledPins[i], LOW);

          Serial.print("Tombol ");
          Serial.print(i + 1);
          Serial.println(" DILEPAS - LED MATI");
        }
      }
    }
  }
}
```

Pertanyaan Analisis

1. **Analisis mengapa debouncing tetap diperlukan pada rangkaian matrix pushbutton ini, walaupun pada satu waktu hanya satu tombol yang biasanya ditekan, kaitkan dengan penyebab bouncing pada C.5 Debouncing.**

   Debouncing tetap diperlukan karena pushbutton merupakan komponen mekanis yang memiliki kontak fisik di dalamnya. Ketika tombol ditekan atau dilepas, kontak tersebut tidak langsung berada pada kondisi stabil, melainkan dapat mengalami getaran kecil (bouncing) yang menyebabkan sinyal berubah-ubah antara HIGH dan LOW dalam waktu singkat.

   Jika tidak menggunakan debouncing, ESP32 dapat membaca satu kali tekanan tombol sebagai beberapa kali perubahan sinyal. Hal ini dapat menyebabkan LED berkedip atau berubah status secara tidak sesuai dengan tekanan tombol. Oleh karena itu, digunakan metode debouncing dengan memberikan waktu tunggu sekitar 50 ms untuk memastikan pembacaan tombol sudah stabil sebelum diproses oleh program.

2. **Analisis konfigurasi pull-up atau pull-down (sesuai rancangan Anda) yang digunakan pada pembacaan tombol, apa kondisi (HIGH/LOW) pin tersebut saat tombol tidak ditekan, dan mengapa kondisi tersebut tidak boleh dibiarkan floating?**

   Pada rangkaian ini digunakan konfigurasi pull-up internal (`INPUT_PULLUP`) pada pin GPIO ESP32. Konfigurasi ini membuat kondisi pin GPIO berada pada HIGH ketika tombol tidak ditekan. Ketika tombol ditekan, pin GPIO akan terhubung ke GND sehingga kondisi pembacaannya berubah menjadi LOW.

   Kondisi tersebut dapat dijelaskan sebagai berikut:

   - Tombol tidak ditekan: HIGH (1).
   - Tombol ditekan: LOW (0).

   Pin GPIO tidak boleh dibiarkan dalam kondisi floating karena tegangan pada pin menjadi tidak memiliki kondisi logika yang pasti. Akibatnya, ESP32 dapat membaca nilai HIGH atau LOW secara acak meskipun tombol tidak sedang ditekan. Penggunaan pull-up internal membantu menjaga kondisi pin tetap stabil sehingga pembacaan tombol menjadi lebih akurat.

3. **Andaikan salah satu tombol pada matrix ini digantikan dengan LDR yang dibaca melalui ADC (seperti Percobaan 5), analisis perbedaan mendasar antara membaca GPIO sebagai digital input (tombol, hanya HIGH/LOW) dan sebagai analog input (ADC), mengapa ADC dapat merepresentasikan lebih dari dua kondisi?**

   Perbedaan mendasar antara digital input dan analog input terletak pada jenis nilai yang dibaca oleh mikrokontroler. Digital input hanya membaca dua kondisi logika, yaitu HIGH (1) dan LOW (0), seperti pada pushbutton yang hanya memiliki kondisi ditekan atau tidak ditekan.

   Sementara itu, analog input menggunakan ADC (Analog to Digital Converter) untuk mengubah tegangan analog menjadi nilai digital. Pada ESP32 dengan resolusi ADC 12-bit, nilai pembacaan secara teoritis dapat berada pada rentang 0–4095.

   Jika tombol digantikan dengan LDR, perubahan intensitas cahaya akan menyebabkan perubahan resistansi LDR dan tegangan pada rangkaian pembagi tegangan. Perubahan tegangan

tersebut kemudian dibaca oleh ADC dan menghasilkan nilai yang berbeda-beda sesuai intensitas cahaya yang diterima.

Dengan demikian, ADC dapat merepresentasikan lebih dari dua kondisi karena mampu membedakan berbagai tingkatan tegangan, bukan hanya HIGH dan LOW seperti digital input.

4 Kesimpulan

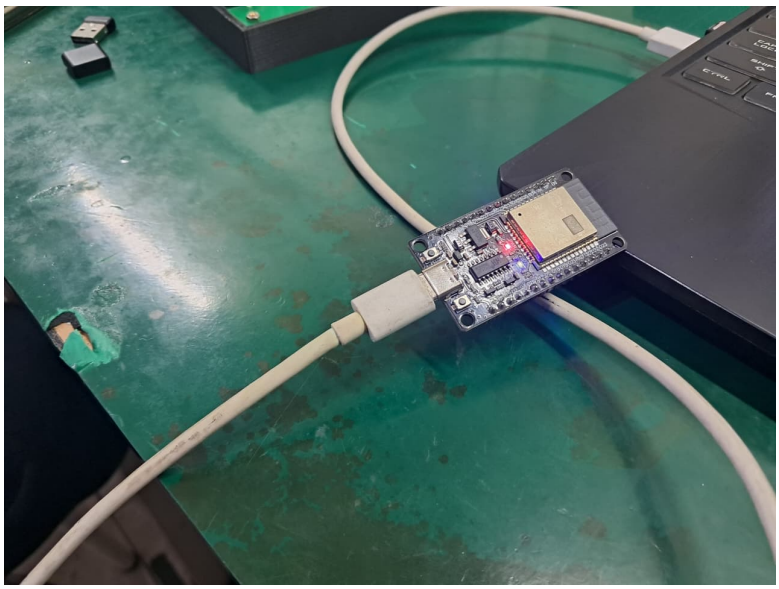
Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa sensor LDR dapat digunakan untuk mendeteksi perubahan intensitas cahaya melalui perubahan nilai resistansi yang memengaruhi tegangan keluaran pada rangkaian pembagi tegangan. ESP32 memanfaatkan fitur Analog-to-Digital Converter (ADC) pada GPIO 34 untuk mengubah tegangan analog menjadi nilai digital dengan resolusi 12-bit atau rentang pembacaan 0 hingga 4095. Melalui pengujian dengan menutup sensor menggunakan tangan dan memberikan pencahayaan dari lampu, diketahui bahwa perubahan intensitas cahaya menghasilkan perubahan nilai ADC yang dapat diamati melalui Serial Monitor. Praktikum ini menunjukkan bahwa sinyal analog mampu merepresentasikan berbagai tingkatan kondisi lingkungan, tidak hanya terbatas pada kondisi HIGH dan LOW seperti pada sinyal digital. Dengan demikian, penggunaan sensor LDR dan ADC pada ESP32 memungkinkan pengukuran intensitas cahaya secara lebih bervariasi dan menjadi dasar dalam pengembangan sistem monitoring maupun otomasi berbasis sensor.

5 Lampiran

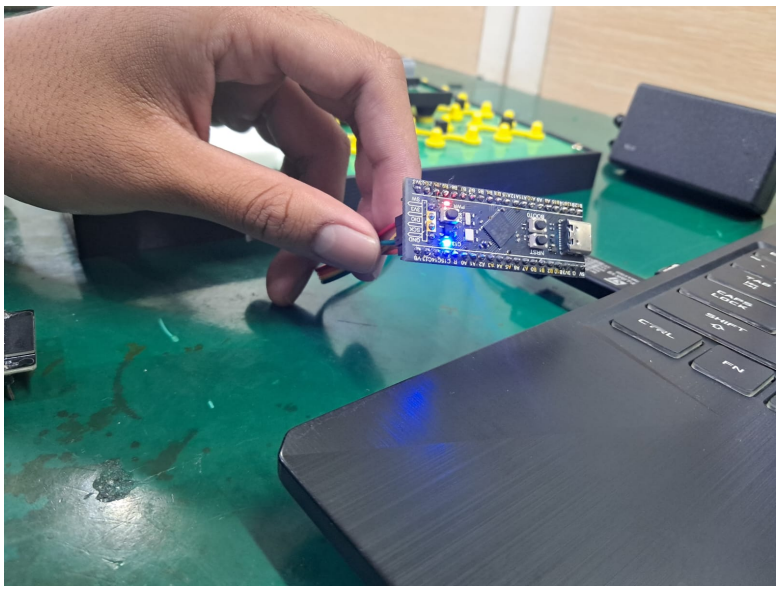
5.1 Dokumentasi saat praktikum



Gambar 25: Kelompok 25



Gambar 26: Lampiran 2



Gambar 27: Lampiran 3